

PRACTICAL : WEEK 8

Task -1:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = malloc(5 * sizeof(int));
    int *q = calloc(5, sizeof(int));

    if (p == NULL || q == NULL)
        return 1;

    for (int i = 0; i < 5; i++)
        p[i] = i + 1;

    p = realloc(p, 10 * sizeof(int));

    if (p == NULL)
    {
        free(q);
        return 1;
    }

    for (int i = 5; i < 10; i++)
        p[i] = i + 1;

    for (int i = 0; i < 10; i++)
        printf("%d ", p[i]);

    printf("\n");

    free(p);
    free(q);

    return 0;
}
```

OUTPUT:

```
[b.deepak@BDeepaks-MacBook-Air ~ % nano memoryalloc.c
[b.deepak@BDeepaks-MacBook-Air ~ % gcc memoryalloc.c -o memoryalloc
[b.deepak@BDeepaks-MacBook-Air ~ % ./memoryalloc
1 2 3 4 5 6 7 8 9 10
b.deepak@BDeepaks-MacBook-Air ~ %
```

Task -2:

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int *data = malloc(100 * sizeof(int));

    if (data == NULL)
        return 1;

    data[0] = 10;

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child before: %d\n", data[0]);
        data[0] = 999;
        printf("Child after: %d\n", data[0]);
        free(data);
    }
    else
    {
        wait(NULL);
        printf("Parent: %d\n", data[0]);
        free(data);
    }

    return 0;
}
```

OUTPUT:

```
[b.deepak@BDeepaks-MacBook-Air ~ % nano cow.c
[b.deepak@BDeepaks-MacBook-Air ~ %
[b.deepak@BDeepaks-MacBook-Air ~ % gcc cow.c -o cow
[b.deepak@BDeepaks-MacBook-Air ~ % ./cow
Child before: 10
Child after: 999
Parent: 10
b.deepak@BDeepaks-MacBook-Air ~ % █
```